

**Carnegie
Mellon
University**

Datasets

Introduction to Deep Learning | 11-485 / 685 / 785

Building the Dataset class, and making it fast enough to keep the GPU busy

Felix Hirwa Nshuti

Some parts adapted from 11875-Spring 2026 slides by Mugur Preda

Fall 2026

What you will leave with

1 The contract

Who owns what.
Dataset answers 'what is one sample'.
DataLoader answers 'how do I iterate'.

2 The HW1P2 data

MFCC frames, transcripts, and
context windows: the exact shapes
you will index into.

3 Making it fast

Eight techniques, each with its
cost. Caching, flattening, batched
fetch, worker tuning.

The one idea to hold on to

Training time per epoch = $\max(\text{GPU compute time}, \text{data pipeline time})$. Optimizing the model is wasted work while the GPU is waiting on your `__getitem__`.

Why Datasets and DataLoaders

Four reasons PyTorch splits these into two objects.

1 Modularization

Dataset abstracts what the data is and how to access one sample.
DataLoader abstracts how to iterate: batching, shuffling, ordering.

2 Efficiency

Load lazily, one batch at a time, instead of all at once.
Overlap loading with compute via `num_workers`.

3 Training support

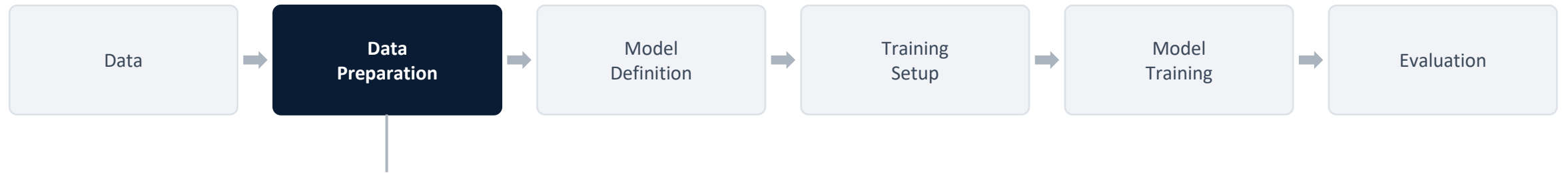
Batching, shuffling and epoch iteration come for free.
This is what makes mini-batch gradient descent practical.

4 Customization

Preprocessing and augmentation live in one place.
Same interface for images, audio, text, EEG, anything.

Where this sits in the training pipeline

Everything today happens in one box, and that box decides your epoch time.



Data Preparation, expanded



The last step is not optional. Load one sample, print its shape and dtype, and plot it before you write a single line of the model. Most HW1P2 debugging sessions end at a shape you never looked at.

Division of labor

Four questions, and which object answers each one.

1

What to read?

`DataLoader` / `Sampler`

Produces the indices for the next batch. `shuffle=True` means a `RandomSampler`.

2

Where to read from?

`Dataset`

Holds the arrays, the file paths, the memory map. Owns the data.

3

How to read one sample?

`Dataset.__getitem__`

Index to tensor. Called once per sample, per epoch. This is the hot path.

4

How to combine samples?

`collate_fn`

Takes a list of samples, returns one batched tensor. Default is `torch.stack`.

The three methods you must write

Plus one optional function that turns out to matter a lot.

```
class MyDataset(torch.utils.data.Dataset):

    def __init__(self, root):          # 1. read paths, load, preprocess, store
        self.x = ...                  # runs ONCE
        self.y = ...

    def __len__(self):                 # 2. how many samples exist
        return len(self.x)            # the Sampler draws indices in [0, len)

    def __getitem__(self, i):          # 3. index -> one (sample, label) pair
        return self.x[i], self.y[i]   # runs len(dataset) times PER EPOCH

    def collate_fn(batch):             # 4. optional: list of samples -> batch
        xs, ys = zip(*batch)          # default is torch.stack
        return torch.stack(xs), torch.stack(ys)
```

Anything expensive belongs in `__init__`, not `__getitem__`. That single rule accounts for most of the speedups in the second half of this talk.

The life of one batch

Six steps between your for-loop and the GPU.

- 1 Sampler yields indices**

RandomSampler if shuffle=True, else sequential. BatchSampler groups them into a list of size batch_size.
- 2 Index list goes to a worker**

One of num_workers subprocesses picks up the list. With num_workers=0 this is the main process, and your GPU idles while it reads.
- 3 __getitem__ runs, once per index**

batch_size Python calls. Every allocation, copy and cast here is paid batch_size times.
- 4 collate_fn merges the list**

List of tensors becomes one tensor with a leading batch dimension.
- 5 pin_memory, then queue**

If pin_memory=True the batch lands in page-locked memory so the host-to-device copy can be async.
- 6 Main process yields the batch**

You call .to(device, non_blocking=True) and the forward pass finally starts.

The HW1P2 data, concretely

Everything you index into for the rest of this talk.

```
x = np.load('mfcc/1272-128104-0000.npy')          # (581, 28)    float32
y = np.load('transcript/1272-128104-0000.npy')    # (583,)      '<U5'

y[:3]    ->    ['[SOS]' '[SIL]' '[SIL]']          # strip [SOS] and [EOS]:  y[1:-1]
```

T Variable length

Each utterance has its own T.
The frame is the training sample,
not the utterance. Frames across
all utterances are one flat pool.

28 Features per frame

MFCC coefficients, float32.
One frame alone carries almost no
phonetic information, which is why
we need context.

T+2 Transcript length

Two extra tokens, [SOS] and [EOS].
Strings, not integers. Map them to
ints once, in `__init__`, never in
`__getitem__`.

Context windows

One frame is not enough. Concatenate the neighbours.

```
context = 20
window  = x[t - context : t + context + 1]    # (41, 28)
sample  = window.reshape(-1)                  # (1148,)  <- MLP input dim
label   = y[1:-1][t]
```

`input_dim = (2 * context + 1) * 28`

Change context and the first Linear layer changes with it. Derive it, do not hardcode 1148.

The boundary problem

At $t = 0$ there is no $t - 20$. Pad with zeros. Pad the whole array once in `__init__`, not per sample, or you pay an allocation on every access.

Never materialize the windows. 28M frames x 41 x 28 x 4 bytes = 128 GB. The window must stay a view into the frame array, computed at access time.

The Dataset everyone writes first

It is correct. That is the only good thing about it.

```
class NaiveDataset(Dataset):
    def __init__(self, root, context):
        self.files = sorted(glob.glob(f'{root}/mfcc/*.npy'))    # paths only
        self.context = context

    def __getitem__(self, i):
        u, t = self.index_map[i]
        arr = np.load(self.files[u])                # (1) disk read, every access
        lo, hi = t - self.context, t + self.context + 1
        w = np.pad(arr, ...)[lo:hi]                # (2) re-pad, every access
        return torch.tensor(w.reshape(-1)), self.labels[i]    # (3) float64 copy
```

- 1 One open() and one parse per sample. 28M syscalls per epoch.
- 2 np.pad allocates a fresh array of the whole utterance, per sample.
- 3 torch.tensor copies. On a Python float it also promotes to float64.

Rule 0: measure before you optimize

Two minutes of measurement beats an hour of guessing.

```
# Is the loader the bottleneck? Iterate it with no model at all.
t0 = time.time()
for x, y in tqdm(train_loader): pass
print(f'data-only epoch: {time.time() - t0:.1f}s')

# Then watch the GPU while training runs:  watch -n0.5 nvidia-smi
```

What you gain

- GPU utilization under 50 percent means you are input-bound. Fix the loader, not the model.
- Utilization pinned near 100 percent means the loader is already keeping up. Stop optimizing it.
- `torch.profiler` with `record_shapes` gives you the per-stage breakdown when you need detail.

What it costs

- Colab shares GPUs, so utilization is noisy. Take a median over a few hundred steps.
- The first epoch is never representative: page cache is cold and workers are still spinning up.
- `tqdm` itself costs time at very small batch sizes. Do not benchmark with it inside a tight loop.

Preload into RAM in `__init__`

Pay the disk cost once per run instead of once per access.

```
def __init__(self, root, context):  
    files = sorted(glob.glob(f'{root}/mfcc/*.npy'))  
    self.utts = [np.load(f) for f in files]      # once, at construction
```

What you gain

- Removes the syscall and the .npy parse from the hot path entirely.
- 20x to 60x faster per sample than re-loading from disk, across our runs.
- Works on Colab, where the disk is a network mount and reads are slow.

What it costs

- Construction now takes seconds to minutes, and you pay it again on every restart.
- The whole split must fit in RAM. train-clean-100 as float32 is about 3.1 GB.
- Dies immediately on datasets larger than memory. Use a memory map instead.

Flatten, and do index arithmetic

Stop asking which utterance a frame belongs to.

```
self.flat      = np.concatenate(self.utts, axis=0)      # (N, 28), one array
self.offsets = np.cumsum([0] + [len(u) for u in self.utts])

# __getitem__ becomes a slice. No lookup, no branch, no searchsorted.
```

What you gain

- One contiguous array means the CPU prefetcher actually works.
- No per-access searchsorted or dict lookup, no Python-level branching.
- Sequential frames land in the same cache lines, so shuffled reads stay cheap.

What it costs

- `np.concatenate` transiently needs double the memory. Free `self.utts` right after.
- Utterance boundaries disappear, so a window can straddle two speakers. For frame classification this is accepted; for anything sequential it is a bug.

Pad once, then return views

The window should be a pointer, not a copy.

```
self.padded = np.pad(self.flat, ((context, context), (0, 0))) # once

def __getitem__(self, i):
    return self.padded[i : i + 2*self.context + 1].reshape(-1), self.y[i]
```

What you gain

- Slicing a numpy array returns a view: no allocation, no memcpy.
- Index i maps straight to row i of the padded array. The boundary case disappears.
- Two orders of magnitude faster per sample than the naive version.

What it costs

- Padding adds 2 x context rows, negligible here but not for large context on small data.
- `reshape(-1)` on a row-slice is free only because the slice is contiguous. Transpose it first and numpy silently copies.

dtype discipline

Half the bytes means half the memory traffic.

```
self.flat = self.flat.astype(np.float32)      # not float64
self.y     = np.array([lut[s] for s in labels], dtype=np.int64)  # ints, once

# torch.from_numpy shares memory. torch.tensor copies. Know which you called.
```

What you gain

- float32 gather measured 1.5x faster than float64, and halves resident memory.
- Labels as int8 instead of '<U5' strings: 20x smaller, and no per-access string hashing.
- GPUs are built for float32. Feeding float64 forces a cast on every batch.

What it costs

- CrossEntropyLoss requires int64 targets, so you cast back at batch level. Cheap, but do not forget it.
- float16 storage is tempting and usually not worth the precision loss on MFCCs.

Caching, three levels

Ordered by what they survive.

```
# L1 in-process dict / functools.lru_cache -> survives the epoch
np.save('cache/train_flat.npy', self.flat) # L2 -> survives the process
self.flat = np.load('cache/train_flat.npy', mmap_mode='r') # L3 -> survives RAM
```

What you gain

- L2 is 5x to 10x faster to construct than re-reading the source files.
- L3 memory-maps: construction is near-instant and the OS page cache is shared across all workers.
- Preprocessing you do once (normalization, label mapping) never runs again.

What it costs

- L1 is per-worker. With num_workers=8 you hold eight copies of the cache.
- L2 goes stale silently. Key the filename on a hash of your preprocessing config.
- L3 is lazy: the first epoch pays page faults, and random access to a cold mmap can be slower than RAM.

Batched fetch, vectorized collate

Stop paying Python overhead `batch_size` times.

```
def __getitem__(self, idxs):          # note the s: PyTorch 2.x calls this
    rows = np.asarray(idxs)[: , None] + self.window      # (B, 2c+1)
    x = self.padded[rows].reshape(len(idxs), -1)         # one gather, no loop
    return torch.from_numpy(x), torch.from_numpy(self.y[idxs])
```

What you gain

- 3x to 7x faster than a Python loop plus `torch.stack`, for a batch of 1024.
- One numpy gather replaces 1024 interpreter round-trips and 1024 small allocations.
- Pairs with `BatchSampler(..., batch_size=B)` and `batch_size=None` on the `DataLoader`.

What it costs

- The gather materializes the batch, so it does copy. That copy is unavoidable and it is vectorized.
- Per-sample random augmentation gets awkward: you now augment the whole batch at once.
- `__getitem__` is a newer API. On older PyTorch, do the same thing in `collate_fn` instead.

DataLoader knobs that actually matter

Four arguments, each with a failure mode.

```
DataLoader(ds, batch_size=1024, shuffle=True,
           num_workers=4,           # overlap loading with compute
           pin_memory=True,         # page-locked -> async host to device
           persistent_workers=True, # do not respawn every epoch
           prefetch_factor=2)       # batches queued per worker
```

What you gain

- `num_workers` hides loading behind compute. On Colab, 2 to 4 is usually the sweet spot.
- `persistent_workers` removes worker startup and cache rebuild from every epoch boundary.
- `pin_memory` plus `.to(device, non_blocking=True)` overlaps the transfer with the forward pass.

What it costs

- Each worker is a process with its own copy of the Dataset. RAM scales with `num_workers`.
- Too many workers thrash. On our single-core box, `num_workers=2` was 40 percent SLOWER than 0. Measure on your runtime.
- `pin_memory` holds non-pageable memory. On a small VM it can trigger the OOM killer.

Two traps worth naming

Both look like memory bugs and neither is.

```
# TRAP A: fork copy-on-write. Python lists get their refcounts touched,  
#         so every worker slowly copies the whole list. numpy arrays do not.  
self.paths = np.array(paths)      # not a Python list of str  
  
# TRAP B: normalize on the GPU, not per sample on the CPU.  
x = (x.to(device, non_blocking=True) - self.mean) / self.std
```

What you gain

- Trap A: storing metadata as numpy arrays keeps worker memory flat instead of growing per epoch.
- Trap B: a batch-level GPU normalize is one kernel instead of batch_size numpy ops on the CPU.
- Both free CPU cycles for the part of the pipeline you cannot vectorize.

What it costs

- Trap A only bites with num_workers > 0 and fork start method. It is invisible on Windows and on Colab TPU runtimes.
- Trap B moves work onto the GPU, which is fine only while the GPU has headroom. Re-measure after you move it.

Measured, on this recitation's data

15 utterances, 14,457 frames, context = 20, single CPU core. Ranges span repeated runs.

Change	What was measured	Speedup
Re-load .npy per access (baseline)	64 samples through <code>__getitem__</code>	1.0x
+ preload into RAM	64 samples through <code>__getitem__</code>	20-60x
+ flatten and pad once, return views	64 samples through <code>__getitem__</code>	~110x
Python loop + <code>torch.stack</code> (baseline)	assemble a batch of 1024	1.0x
+ vectorized gather (<code>__getitem__</code>)	assemble a batch of 1024	3-7x
float64 -> float32	same gather, half the bytes	1.5x
Re-read source files (baseline)	Dataset construction	1.0x
+ single cached .npy	Dataset construction	5-10x
'<U5' labels -> int8	label memory footprint	20x smaller

Relative numbers, not absolute ones. Re-run the benchmark cell in Part 3 on your own runtime before you trust any of them.

Which technique, when

Do not apply all eight. Apply the ones your bottleneck justifies.

Does the split fit in RAM?

Yes: preload and flatten (1, 2, 3). No: memory-map the cache (5, level 3).

Is GPU utilization below 50 percent?

Yes: keep going down this list. No: you are compute-bound. Stop and go tune the model.

Is construction the slow part?

Cache the preprocessed array to disk (5, level 2). Key it on your config.

Is per-sample overhead the slow part?

Batched fetch (6). This is where a 1024-sample batch pays off.

Are you using augmentation?

Cache the raw array, not the augmented one. Augment in `collate_fn` or on the GPU.

Still slow, and RAM is climbing?

Check trap A: Python containers under fork (8).

Before you optimize: stay correct

Every technique here can silently break your labels.

What you gain

- Compare against the naive Dataset: same index, same tensor, element for element. Write the assert.
- shuffle=True on train only. Never on validation or test, or your predictions will not line up with the CSV.
- Strip [SOS] and [EOS] once, in __init__. Assert len(y) == len(x) afterwards.
- Fit normalization statistics on train only, then reuse them for val and test.

What it costs

- A cache built before you changed context is still on disk and will load without complaint.
- Caching augmented samples destroys the randomness that augmentation exists to provide.
- A window straddling an utterance boundary is fine for frame classification and wrong for sequence tasks.
- A wrong offset in fancy indexing yields plausible tensors with shifted labels. It trains, badly.

The test that catches all of this

```
for i in [0, 1, len(fast)//2, len(fast)-1]:
    xf, yf = fast[i]; xn, yn = naive[i]
    assert torch.allclose(xf, xn) and yf == yn
```

Where to go next

Everything below is free and worth the time.

- **Notebooks for this recitation** Part 1 (Dataset basics), Part 2 (HW1P2 data), Part 3 (optimization, new this semester). Linked on the course site.
- **PyTorch data tutorial** pytorch.org/tutorials/beginner/basics/data_tutorial.html
- **DataLoader API reference** pytorch.org/docs/stable/data.html - read the notes on multi-process loading and memory pinning
- **torch.profiler recipe** pytorch.org/tutorials/recipes/recipes/profiler_recipe.html
- **numpy views vs copies** numpy.org/doc/stable/user/basics.copies.html - the mental model behind techniques 3 and 6
- **Ask on Piazza** Post your data-only epoch time and your GPU utilization. Those two numbers make the answer obvious.

Thank you

Write the naive version first. Measure. Then earn every optimization.

Questions on Piazza | Recitation 0 notebooks available on the course site